FPT Education
FPT UNIVERSITY

Chapter 9: File Systems

VFS • Flash FS • Embedded Storage

Learning Objectives

- Understand Linux file system concepts
- Learn Virtual File System (VFS)
- Explore flash file systems
- Understand mounting and storage

FPT Education
FPT UNIVERSITY

File System Concepts

- Block devices store fixed-size data blocks
- Partitions divide storage into logical units
- VFS provides unified interface:
  - superblock: represents a mounted filesystem
  - inode: represents a file (metadata + identity)
  - dentry: maps path names to inodes (directory entry)
  - file: represents an open file instance (runtime state)

Khi Linux mount một filesystem, kernel tạo/duy trì một struct super_block đại diện cho filesystem đó trong VFS.

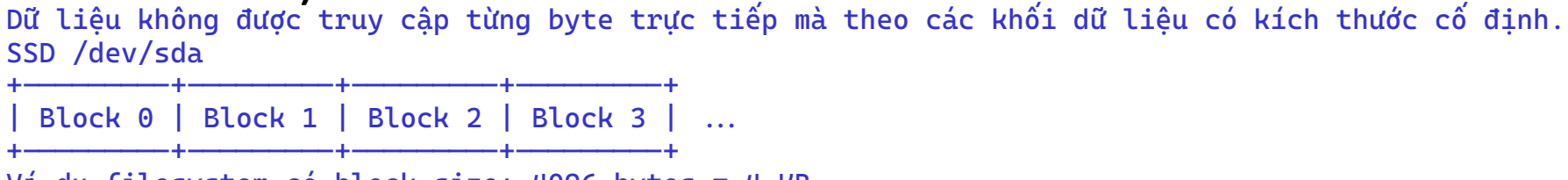
Một inode đại diện cho đối tượng filesystem thực sự.

directory entry

một file đang được mở

```
int fd = open("test.txt", O_RDONLY);
```

Dữ liệu không được truy cập từng byte trực tiếp mà theo các khối dữ liệu có kích thước cố định.



Ví dụ filesystem có block size: 4096 bytes = 4 KB

```
partition:
/dev/sda
   |
   +-- /dev/sda1    EFI
   |
   +-- /dev/sda2    ext4
   |
   +-- /dev/sda3    swap
```

```
/dev/sdb1
    |
    | FAT32/ext4
    v
+--------------------+
| struct super_block |
+--------------------+
         |
      /mnt/usb
```

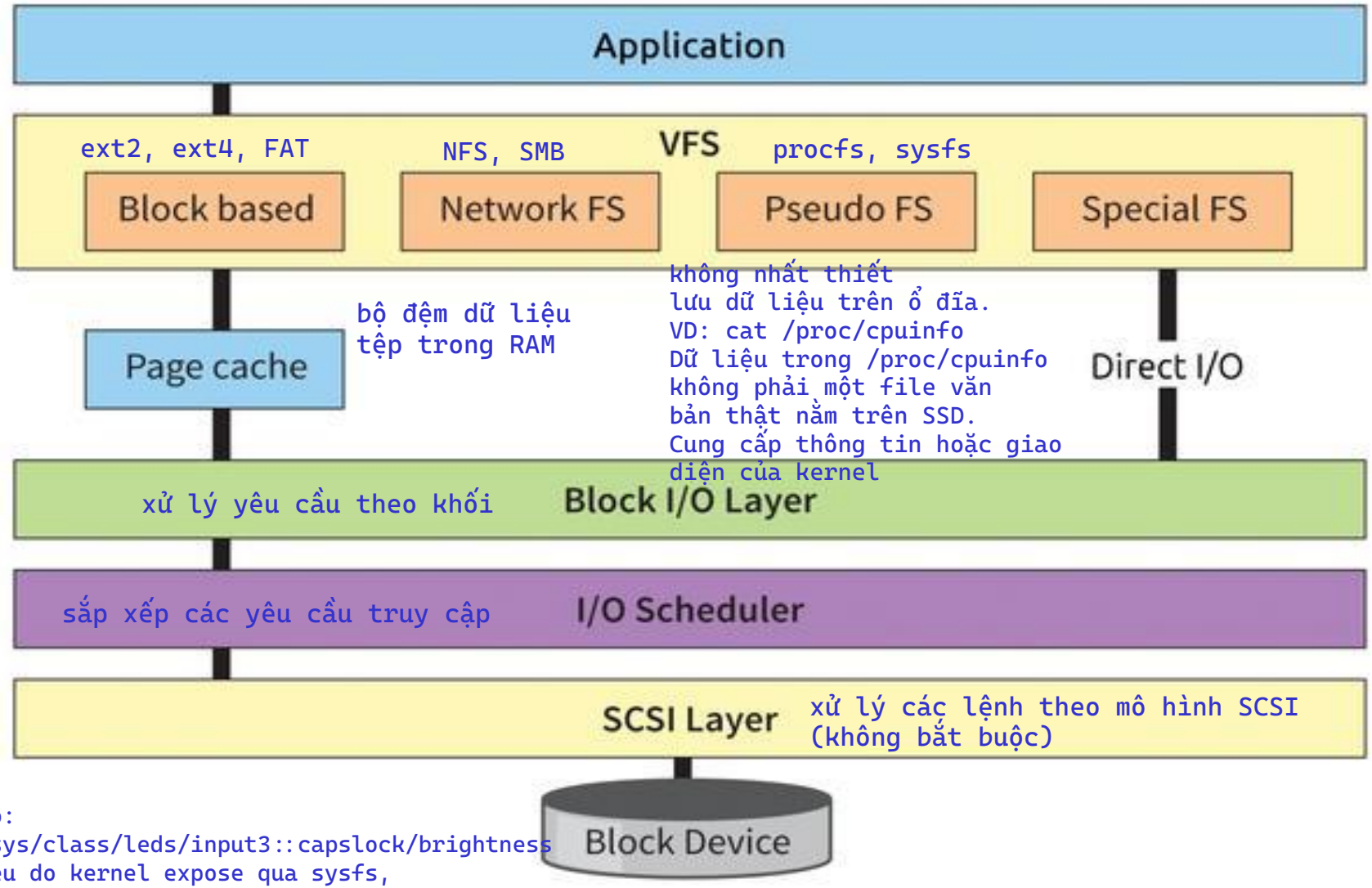
```
dentry "/"
   |
dentry "home"
   |
dentry "vinh"
   |
dentry "test.txt"
   |
   v
inode 15231
```

VD inode:
Ví dụ file:
/home/user/test.txt
inode = 15231
ls -li test.txt
Inode chứa metadata như:
file type
permissions
owner UID/GID
file size
timestamps
link count
location/mapping của data blocks

```
dentry
   |
   +---- name = "test.txt"
   |
   +---- inode ------> inode 15231
```

FPT Education
FPT UNIVERSITY

File System Diagram



pseudo:
cat /sys/class/leds/input3::capslock/brightness
dữ liệu do kernel expose qua sysfs,
không phải dữ liệu file bình thường trên disk.

FPT Education
FPT UNIVERSITY

không có journaling, tức không có cơ chế nhật ký giao dịch để hỗ trợ phục hồi các thay đổi filesystem khi hệ thống bị gián đoạn.

ext2 Filesystem

Journal chủ yếu bảo vệ tính nhất quán của metadata; không đồng nghĩa mọi nội dung vừa ghi đều được bảo toàn.

- Lightweight, no journaling
- Raw NAND (no FTL): use flash-aware FS (UBIFS, YAFFS2)
- eMMC / SD (with FTL): ext4/ext2 acceptable
- NOR flash: small systems may use JFFS2 or even ext2 (limited cases)
- Filesystem must match storage characteristics
- Requires fsck after crash

NAND thô cần filesystem hiểu flash.

Flash Translation Layer là lớp ánh xạ địa chỉ khối logic mà hệ điều hành nhìn thấy sang vị trí vật lý trên flash.

Không nên xem ext2 là lựa chọn mặc định cho raw NOR có ghi thường xuyên.

fsck là công cụ kiểm tra và sửa tính nhất quán của filesystem.
Với ext2, việc không được unmount sạch là một điều kiện khiến filesystem cần được kiểm tra.

- ? ext2 is fast but risky without journaling

không nên hiểu là ext2 luôn nhanh hơn ext4; tốc độ còn phụ thuộc tác vụ và thiết bị.

journal, ghi nhận các giao dịch cập nhật filesystem:
1. Ghi thông tin cập nhật cần bảo vệ vào journal.
2. Đánh dấu giao dịch đã hoàn tất (commit).
3. Đưa các cập nhật đến vị trí chính thức trong filesystem.

Nếu mất điện sau bước 2 nhưng trước khi bước 3 hoàn thành,
hệ thống có thể phát lại giao dịch đã commit để hoàn tất cập nhật.
Giao dịch chưa commit không được xem là hoàn tất.

ext3 & ext4

- ext3 adds journaling
  quản lý theo dải block liên tiếp
- ext4 uses extents and delayed allocation
  trì hoãn cấp phát block
- ext4 is default in modern Linux
  xt4 là một trong những filesystem phổ biến nhất trên Linux

 ext4 balances performance, reliability, and scalability

Một extent mô tả một dải block dữ liệu nằm liên tiếp trên thiết bị lưu trữ.
Ví dụ, file có 100 block nằm liên tục từ block vật lý 5000 đến 5099:
Cách dùng con trỏ block của ext2/ext3: Vị trí từng block: 5000, 5001, …, 5099
Extent của ext4: Block logic bắt đầu = 0; block vật lý bắt đầu = 5000; số block = 100

Thay vì lưu nhiều con trỏ riêng lẻ, ext4 có thể mô tả cả dải bằng một extent.
Điều này giảm metadata cần lưu và xử lý, đặc biệt có lợi cho file lớn có dữ liệu liên tiếp. Nếu file bị phân mảnh,

Khi chương trình gọi write(), dữ liệu thường được đưa vào page cache trong RAM trước.
Với delayed allocation, ext4 chưa chọn ngay các block vật lý cuối cùng cho dữ liệu đó;
nó trì hoãn quyết định đến lúc cần ghi dữ liệu xuống thiết bị.

Ví dụ, chương trình ghi một file thành ba lần, mỗi lần 4 KB:
- Nếu chọn vị trí ngay từng lần, các phần dữ liệu có thể được cấp phát rải rác.
- Nếu chờ gom đủ 12 KB, filesystem có cơ hội chọn một vùng trống liên tiếp cho cả phần dữ liệu.

Cách này giúp giảm phân mảnh và cải thiện quyết định cấp phát.
Nó kết hợp tốt với extent: delayed allocation giúp tạo các dải liên tiếp,
còn extent giúp mô tả các dải đó gọn hơn.

Legacy File Systems

- ReiserFS uses:
  - Designed for efficient storage of many small files
  - Uses B-tree structure for fast lookup and metadata handling
  - Tail packing: stores small file tails inside blocks to save space Tail là phần dữ liệu cuối file không đủ lấp đầy một block.
- ReiserFS deprecated in modern kernels
  - Performance alone is not enough → maintenance matters
  - Filesystem must evolve with kernel ecosystem
  - Community support is critical for long-term viability

Ví dụ, giả sử block có kích thước 4 KiB = 4096 byte, mỗi file chỉ chứa 100 byte:
- Nếu mỗi file chiếm riêng một block, phần chưa sử dụng là 4096−100=3996 byte.
- Với 10.000 file, dung lượng block dữ liệu là khoảng 39 MiB, trong khi nội dung thực tế chỉ khoảng 0,95 MiB.

B-tree là cấu trúc cây tìm kiếm trong đó một node có thể chứa nhiều khóa và nhiều nhánh con.
Khi cần tìm thông tin của một file, filesystem:
1. Đọc node gốc.
2. Dựa vào khóa tìm kiếm để chọn nhánh phù hợp.
3. Đi xuống các node tiếp theo đến node lá chứa thông tin cần tìm.

Ví dụ tail packing, block có kích thước 4096 byte và file dài 5000 byte:
5000=4096+904

	Block đầy đầu tiên	Phần 904 byte cuối
Cấp phát riêng từng block	Chiếm một block	Chiếm thêm một block, còn trống 3192 byte
Tail packing	Chiếm một block	Có thể đóng gói trong node lá cùng các item khác

ReiserFS có thể lưu phần đuôi dưới dạng dữ liệu nhỏ trong các node lá của cây,
chia sẻ không gian với những item khác.

JFFS2

JFFS2 phải dựng lại chỉ mục trong RAM khi mount, nên dung lượng flash càng lớn thì quá trình này càng tốn thời gian.

- JFFS2 scans entire flash at mount to rebuild in-memory index
- No on-flash index → mount time = O(n) with flash size
- Large flash → very slow boot (not scalable)
- Designed for raw flash via MTD layer (NOR / small NAND)
- Not usable on block devices (eMMC / SD)Suitable: small NOR / small NAND systems
- Avoid: large NAND or systems requiring fast boot

Memory Technology Device là lớp trong Linux cung cấp giao diện truy cập flash thô, bao gồm các thao tác đọc, ghi và xóa eraseblock.

JFFS2 không dùng trực tiếp trên thiết bị khối thông thường như ext4.

→ Key insight: JFFS2 trades simplicity for poor scalability

JFFS2 → MTD → driver flash → chip NOR/NAND

Khi mount, JFFS2 cần đọc thông tin trên phân vùng flash chứa filesystem để xác định:
- Những file và thư mục nào đang tồn tại.
- Những node nào còn hợp lệ, node nào đã lỗi thời.
- Các đoạn dữ liệu của mỗi file nằm ở đâu.
- Vùng nào còn trống hoặc cần thu hồi.

Compressed FS

dữ liệu được lưu ở dạng nén trên thiết bị; khi đọc, kernel giải nén phần cần dùng vào RAM, không cần giải nén toàn bộ filesystem trước. Cả cramfs và SquashFS đều là filesystem chỉ đọc.

- cramfs (Legacy)
  - Early compressed filesystem with limited features; now considered obsolete and no longer recommended for modern systems.
- squashfs (Modern)
  - High-performance compressed filesystem supporting large files, better compression ratios, and efficient read-only access.
  - # Create image from rootfs directory
    - mksquashfs rootfs/ rootfs.sqsh -comp xz -b 1M -noappend
  - # Mount on target
    - mount -t squashfs rootfs.sqsh /mnt
- Usage
  - Commonly used for read-only root filesystems in embedded systems and firmware images.

FPT Education
FPT UNIVERSITY

EXT4, SquashFS, JFFS2...

Feature	EXT4	F2FS	JFFS2	SquashFS
Type	Block FS	Block FS (flash-aware)	Flash FS (MTD)	Read-only FS
Storage	eMMC / SD (FTL)	eMMC / SD (FTL)	Raw NOR / NAND	Any (image)
Read/Write	Read/Write	Read/Write	Read/Write	Read-only
Optimization	General-purpose	Flash-friendly (reduce write amplification)	Flash-aware (wear leveling, log-structured)	High compression
Mount Time	Fast	Fast	Slow (O(n) scan)	Fast
Scalability	High	High	Poor (large flash)	High
Compression	No (optional via fs-level tools)	No (optional)	No	Yes (built-in)
Typical Use	General Linux systems	Modern embedded (Android, eMMC)	Small raw flash systems	Firmware / rootfs
Limitations	Not flash-aware	Needs FTL	Slow boot, not for large NAND	No runtime write

FPT Education
FPT UNIVERSITY

dùng thư mục trên máy tính làm hệ thống tệp gốc (rootfs) cho board Linux thông qua mạng

Network File System (NFS)

- Mount remote filesystem
  chỉ mount thêm thư mục chia sẻ, chẳng hạn tại /mnt/share
- Useful for development
  mục trên máy chủ trở thành / của board (/bin, /etc, /lib, /sbin)
- Supports rootfs over network
  máy chủ /nfsroot/sbin/init ⇒ board /sbin/init
- Host Configuration (/etc/exports)
  - /nfsroot 192.168.1.0/24(rw,sync,no_root_squash,no_subtree_check)
    Thư mục trên máy chủ được chia sẻ
    thư mục nào được chia sẻ, máy nào được truy cập và với quyền gì.
    Cho phép các client thuộc mạng 192.168.1.x truy cập
    Giữ nguyên danh tính root từ client, thay vì ánh xạ thành người dùng ít quyền
    Tắt kiểm tra cây thư mục con khi phục vụ các tệp được chia sẻ
  - exportfs -ra
    đọc lại và cập nhật toàn bộ cấu hình export; -r là cập nhật lại, -a là áp dụng cho tất cả.
  - systemctl restart nfs-kernel-server
    khởi động lại dịch vụ NFS server

FPT Education
FPT UNIVERSITY

Network File System (NFS)

- Kernel Boot Parameters
  - root=/dev/nfs Dùng NFS làm rootfs
  - nfsroot=192.168.1.1:/nfsroot rootfs Nằm trong /nfsroot trên máy chủ 192.168.1.1
  - ip=dhcp Kernel yêu cầu cấu hình mạng qua DHCP

/dev/nfs là giá trị đặc biệt để báo cho kernel dùng NFS root,
không phải một phân vùng hay tệp thiết bị cần tự tạo

setenv bootargs root=/dev/nfs nfsroot=192.168.1.1:/nfsroot ip=dhcp

boot

quy trình khởi động:
1. U-Boot nạp kernel và DTB vào RAM, rồi truyền bootargs cho kernel.
2. Kernel khởi tạo phần cứng và mạng, sau đó lấy IP theo ip=dhcp.
3. Kernel kết nối máy chủ 192.168.1.1, yêu cầu mount /nfsroot làm /.
4. Kernel chạy tiến trình init, thường là /sbin/init, từ rootfs vừa mount.
5. Các chương trình trên board tiếp tục đọc/ghi tệp trong rootfs thông qua NFS.

FPT Education
FPT UNIVERSITY

Pseudo File Systems

các hệ thống tệp không cần một phân vùng đĩa riêng, dùng để cung cấp thông tin kernel, truy cập thiết bị hoặc lưu dữ liệu tạm thời.

- /proc: Virtual filesystem exposing runtime kernel data such as processes, memory usage, CPU info, and system statistics.
  /proc và /sys chủ yếu cung cấp giao diện với kernel
- /sys: Provides a structured view of devices, drivers, and buses based on the Linux kernel device model.
- devtmpfs (for /dev) tạo các nút thiết bị trong /dev
  - Kernel auto-creates device nodes
  - Mounted on /dev very early in boot
  - Foundation for udev / mdev to apply naming/permissions
- tmpfs (in-memory FS)
  - Stored in RAM (can swap if enabled)
  - Used for /tmp, /run, shared memory
  - Fast, volatile (data lost on reboot)

tmpfs thực sự lưu nội dung tệp do chương trình tạo ra.

/sys/devices/ Cấu trúc phân cấp các thiết bị
/sys/bus/ Tổ chức theo bus, như USB, PCI, I²C
/sys/class/ Tổ chức theo chức năng, như mạng, LED, terminal
/sys/module/ Thông tin và tham số của các module

- Usage: Essential for system monitoring, debugging, and interacting with kernel parameters at runtime.

```
cat /proc/meminfo
cat /proc/1/status
```

/proc: thiên về tiến trình và trạng thái hệ thống.
/sys: thiên về các đối tượng kernel, thiết bị và thuộc tính của chúng.

Modern Embedded FS

- UBIFS: Flash-aware filesystem for NAND devices; supports wear leveling, journaling, and scales efficiently on large storage.
- F2FS (Flash-Friendly FS): Flash-Friendly File System, tối ưu cho flash có bộ điều khiển
  - Read/write FS optimized for eMMC / SD (FTL devices)
  - Log-structured → reduces write amplification
  - Widely used in Android, Raspberry Pi OS
- EROFS (Enhanced Read-Only FS): hệ thống tệp chỉ đọc, hỗ trợ nén
  - Read-only compressed FS (next-gen SquashFS alternative)
  - Faster mount, better random read performance
  - Used in Android system partitions, modern embedded firmware
- Overlayfs: Layered filesystem combining read-only and writable layers; widely used in containers and embedded rootfs designs.

Building Filesystem

- Use dd to create image
  - # Tạo ext4 image
  - dd if=/dev/zero of=rootfs.img bs=1M count=64
  - mkfs.ext4 rootfs.img
  - # Tạo squashfs image
  - mksquashfs rootfs/ rootfs.sqsh -comp xz
  - # Tạo UBIFS image
  - mkfs.ubifs -r rootfs/ -m 2048 -e 131072 -c 512 -o rootfs.ubifs
- Format using mkfs
- Mount using loop device
  - mount -o loop rootfs.img /mnt

Summary

- VFS enables multiple filesystem support
- ext4 is default general-purpose FS
- squashfs + UBIFS dominate embedded systems
- pseudo FS provides system introspection